Linux System Programming

GNU Toolchain

(Linux Development Environment)

Authored and Compiled By: **Boddu Kishore Kumar**

Email: kishore@kernelmasters.com

Reach us online: www.kernelmasters.com

Contact: 9949062828

Important Notice

This courseware is both the product of the author and of freely available open source materials. Wherever external material has been shown, it's source and ownership have been clearly attributed. We acknowledge all copyrights and trademarks of the respective owners.

The contents of this courseware cannot be copied or reproduced in any form whatsoever without the explicit written consent of the author.

Only the programs - source code and binaries (where applicable) - that form part of this courseware, and that are present on the participant CD, are released under the GNU GPL v2 license and can therefore be used subject to terms of the afore-mentioned license. If you do use any of them, in any manner, you will also be required to clearly attribute their original source (author of this courseware and/or other copyright/trademark holders).

The duration, contents, content matter, programs, etc. contained in this courseware and companion participant CD are subject to change at any point in time without prior notice to individual participants.

Care has been taken in the preparation of this material, but there is no warranty, expressed or implied of any kind and we can assume no responsibility for any errors or omisions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

Embedded Systems Development Environement

Windows Platfrom:

- Keil
- Code Composer studio
- IAR Workbench

Linux Platfrom:

- GNU toolchain

Linux Development Enviornment (GNU Tool Chain)

GNU:

GNU is a recursive acronym for *"**GNU's Not Unix**!"*, chosen because GNU's design is Unix-like, but differs from Unix by being free software and containing no UNIX code.

GNU is an operating system and an extensive collection of computer software. GNU is composed wholly of free software, most of which is licensed under the GNU Project's own GPL.

GNU Tool Chain:

The GNU toolchain is a broad collection of programming tools produced by the GNU Project. These tools form a toolchain used for developing software applications and operating systems.

The GNU toolchain plays a vital role in development of Linux, some BSD systems, and software for **embedded systems**.

1. GCC (GNU 'C' Compiler (or) GNU Compiler Collection)
2. GDB (GNU Debugger) - a code debugging tool
3. GNU make - an automation tool for compilation and build
4. GNU Binutils - a suite of tools including linker, assembler and other tools
5. GNU Build System – a configuraiton tool

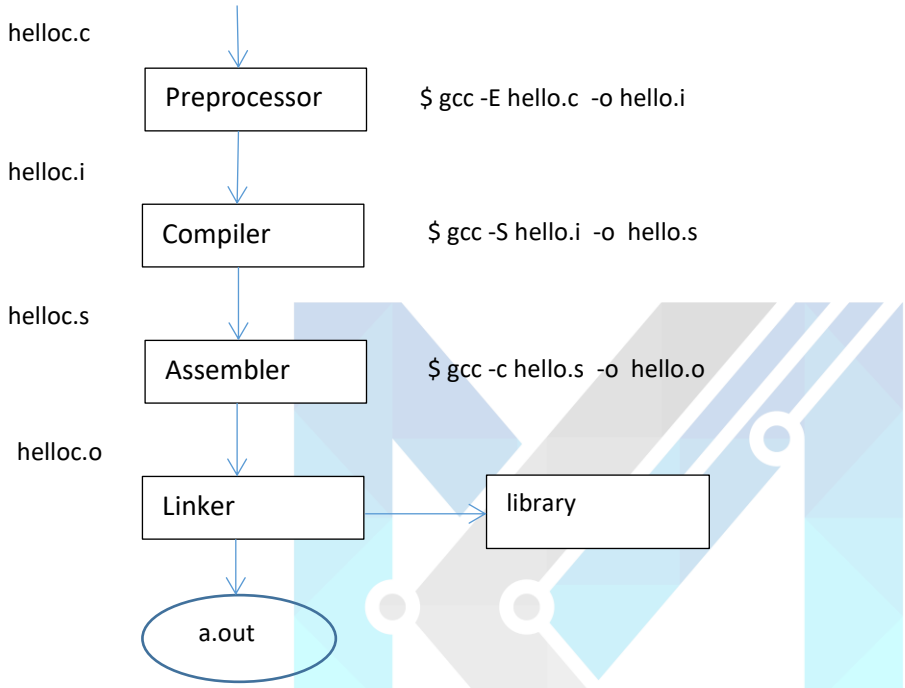
1. GCC (GNU 'C' Compiler (or) GNU Compiler Collection) :

GCC Command line options:

-o	output
-E	Stop after the preprocessing stage.
-S	Stop after the compilation stage.
-c	Stop after the assembler stage.
-g	enable debugging symbol
-I <user defined header file path>	
-L <user defined library path>	
-l<library_name>	
-v	Print the commands executed to run the stages of compilation.
-Wall	enable all warnings
-Werror	Convert warnings into errors

-D[MACRO] Define a MACRO
-U[MACRO] Undefine a MACRO
@file Read command line options from file. Options in file are separated by whitespace.
-funsigned-char char type is treated as unsigned type.
-fsigned-char char type is treated as signed type.

C Programming Compilation (GCC) Stages:



Examples:

+ hello/hello.c - compilation stages (preprocessor, compiler, assembler, linker)
+ addsub/main.c,sub.c,add.c,myinclude.h - How to compile multiple source files.

2. GDB (GNU Debugger):

- A debugger for several languages, including C and C++
- It allows you to inspect what the program is doing at a certain point during execution.
- Errors like segmentation faults may be easier to find with the help of gdb.
- http://sourceware.org/gdb/current/onlinedocs/gdb toc.html - online manual.

Additional step when compiling program:
Normally, you would compile a program like:
gcc [flags] <source files> -o <output file>
For Example
gcc -Wall -Werror hello.c -o hello.x

Now you add a **-g** option to enable built-in debugging support (which gdb needs):

*gcc [other flags] **-g** <source files> -o <output file>*
For Example:
*gcc -Wall -Werror **-g** hello.c -o hello*

Starting up gdb
Just try "gdb" or "gdb hello" You'll get a prompt that looks like this:
(gdb)
If you didn't specify a program to debug, you'll have to load it in now:
(gdb) file hello
Here, hello is the program you want to load, and "file" is the command to load it.
Gdb help
If you're ever confused about a command or just want more information, use the "help" command, with or without an argument:
(gdb) help [command]
Running the Program
(gdb) run
Settng Breakpoints
Breakpoints can be used to stop the program run in the middle, at a designated point. The simplest way is the command "break."
This sets a breakpoint at a specified file-line pair:
(gdb) break file1.c:6
This sets a breakpoint at line 6, of file1.c. Now, if the program ever reaches that location when running, the program will pause and prompt you for another command.

"You can set as many breakpoints as you want, and the program should stop execution if it reaches any of them."

Once you've set a breakpoint, you can try using the run command again. This time, it should stop where you tell it to (unless a fatal error occurs before reaching that point).
You can proceed onto the next breakpoint by typing "continue" (Typing run again would restart the program from the beginning, which isn't very useful.)
(gdb) continue
You can single-step (execute just the next line of code) by typing "step." This gives you really fine-grained control over how the program proceeds. You can do this a lot...
(gdb) step
Similar to "step," the "next" command single-steps as well, except this one doesn't execute each line of a sub-routine, it just treats it as one instruction.
(gdb) next
Print Variables:
The print command prints the value of the variable specified, and print/x prints the value in hexadecimal:
(gdb) print my var
(gdb) print/x my var
Setting Watchpoints:
Whereas breakpoints interrupt the program at a particular line or function, watchpoints act on variables. They pause the program whenever a watched variable's value is modified. For example, the following watch command:

(gdb) watch my_var

backtrace - produces a stack trace of the function calls that lead to a seg fault (should remind you of Java exceptions)
where - same as backtrace; you can think of this version as working even when you're still in the middle of the program
finish - runs until the current function is finished
delete - deletes a specified breakpoint
info breakpoints - shows information about all declared breakpoints

 Step In	 Step Out	 Step Over
(gdb) s (step)	(gdb) n (next)	(gdb) f (finish)

GDB Example:
gdb/General_Debug - sample debugging examples.

Memory Segmentation:

A typical memory representation of C program consists of following sections.
1. Text segment
2. Initialized data segment
3. Uninitialized data segment
4. Stack
5. Heap

A typical memory layout of a running process

1. Text Segment:
A text segment , also known as a code segment or simply as text, is one of the sections of a program in an object file or in memory, which contains executable instructions.

As a memory region, a text segment may be placed below the heap or stack in order to prevent heaps and stack overflows from overwriting it.

Usually, the text segment is sharable so that only a single copy needs to be in memory for frequently executed programs, such as text editors, the C compiler, the shells, and so on. Also, the text segment is often read-only, to prevent a program from accidentally modifying its instructions.

2. Initialized Data Segment:
Initialized data segment, usually called simply the Data Segment. A data segment is a portion of virtual address space of a program, which contains the global variables and static variables that are initialized by the programmer.

Note that, data segment is not read-only, since the values of the variables can be altered at run time.

This segment can be further classified into initialized read-only area and initialized read-write area.

For instance the global string defined by char s[] = “hello world” in C and a C statement like int debug=1 outside the main (i.e. global) would be stored in initialized read-write area. And a global C statement like const char* string = “hello world” makes the string literal “hello world” to be stored in initialized read-only area and the character pointer variable string in initialized read-write area.

Ex: static int i = 10 will be stored in data segment and global int i = 10 will also be stored in data segment

3. Uninitialized Data Segment:
Uninitialized data segment, often called the “bss” segment, named after an ancient assembler operator that stood for “block started by symbol.” Data in this segment is initialized by the kernel to arithmetic 0 before the program starts executing

uninitialized data starts at the end of the data segment and contains all global variables and static variables that are initialized to zero or do not have explicit initialization in source code.

For instance a variable declared static int i; would be contained in the BSS segment.
For instance a global variable declared int j; would be contained in the BSS segment.

4. Stack:
The stack area traditionally adjoined the heap area and grew the opposite direction; when the stack pointer met the heap pointer, free memory was exhausted. (With modern large address spaces and virtual memory techniques they may be placed almost anywhere, but they still typically grow opposite directions.)

The stack area contains the program stack, a LIFO structure, typically located in the higher parts of memory. On the standard PC x86 computer architecture it grows toward address zero; on some other architectures it grows the opposite direction. A “stack pointer” register tracks the top of the stack; it is adjusted each time a value is “pushed” onto the stack. The set of values pushed for one function call is termed a “stack frame”; A stack frame consists at minimum of a return address.

Stack, where automatic variables are stored, along with information that is saved each time a function is called. Each time a function is called, the address of where to return to and certain information about the caller’s environment, such as some of the machine registers, are saved on the stack. The newly called function then allocates room on the stack for its automatic and temporary variables. This is how recursive functions in C can work. Each time a recursive function calls itself, a new stack frame is used, so one set of variables doesn’t interfere with the variables from another instance of the function.

5. Heap:
Heap is the segment where dynamic memory allocation usually takes place.

The heap area begins at the end of the BSS segment and grows to larger addresses from there.The Heap area is managed by malloc, realloc, and free, which may use the brk and sbrk system calls to adjust its size (note that the use of brk/sbrk and a single “heap area” is not required to fulfill the contract of malloc/realloc/free; they may also be implemented using mmap to reserve potentially non-contiguous regions of virtual memory into the process’ virtual address space). The Heap area is shared by all shared libraries and dynamically loaded modules in a process.

Examples:

The size(1) command reports the sizes (in bytes) of the text, data, and bss segments. (for more details please refer man page of size(1))

In Linux,show memory segmentation of a program:
$ cat /proc/<pid>/maps

X86 Stack Frame

When a function is called, the stack frame is formed as follows (grows from higher addresses toward lower addresses; the colour-coding below is just a visual aid):

1. Parameters are pushed onto the stack in reverse order (because of the LIFO mechanism).
2. When the assembly language "call" instruction is issued, to change the execution context to the called function, the return address is pushed onto the stack. This will be the address of the instruction following the current EIP.
3. (Procedure Prolog): Current value of EBP (the frame pointer) is pushed onto the stack. This value is called the Saved Frame Pointer (SFP) and is later used to restore EBP back to it's original state. The current value of ESP is then copied into EBP to set the new frame pointer.
4. Memory is allocated onto the stack for local (automatic) variables by subtracting from ESP. The memory allocated for these variables isn't pushed onto the stack, so the variables are in expected (same) order (as their being declared).

So, basically, we can summarize the stack frame as being formed like this:

```
[... <-- Top (ESP); lower addresses.
LOCALS
...]
SFP <-- EBP
RET addr
[...
PARAMS
...] <-- Bottom; higher addresses.
```

Example: Prepare a x86 stack frame the below program.

stack.c

```
include<stdio.h>

void func1(int , int);

main()
{
int i=3;
func1(1,2);
}

void func1(int a, int b)
{
int x;
x=a;
}
```

Step1: Assign a breakpoint at func1()

Step2: Execute run command in gdb prompt.

Step3: Run disass command and see assembly code.

Step4: Run the below command in gdb prompt and prepare a stack frame format.

32 bit OS:

(gdb) x/20xh $ebp-0x30

64 bit OS:

(gdb) x/20xg $rbp-0x30

```
$ gcc -g stack.c -o stack
$ gdb ./stack
……..
Reading symbols from ./stack...done.
(gdb) l
warning: Source file is more recent than executable.
1       #include<stdio.h>
2
3       void func1(int , int);
4
5       main()
6       {
7       int i=3;
8       func1(1,2);
9       }
10
(gdb) l
11      void func1(int a, int b)
12      {
13              int x;
14              x=a;
15      }
16
17
(gdb) r
Starting program: /home/kernel/KM_GIT/cvital/Stack_Segment/src/stack
Breakpoint 1, func1 (a=1, b=2) at stack.c:14
14      x=a;
```

```
(gdb) disass
Dump of assembler code for function func1:
   0x000000000040050d <+0>:     push   %rbp
   0x000000000040050e <+1>:     mov    %rsp,%rbp
   0x0000000000400511 <+4>:     mov    %edi,-0x14(%rbp)
   0x0000000000400514 <+7>:     mov    %esi,-0x18(%rbp)
=> 0x0000000000400517 <+10>:    mov    -0x14(%rbp),%eax
   0x000000000040051a <+13>:    mov    %eax,-0x4(%rbp)
   0x000000000040051d <+16>:    pop    %rbp
   0x000000000040051e <+17>:    retq
End of assembler dump.
(gdb) disass main
Dump of assembler code for function main:
   0x00000000004004ed <+0>:     push   %rbp
   0x00000000004004ee <+1>:     mov    %rsp,%rbp
   0x00000000004004f1 <+4>:     sub    $0x10,%rsp
   0x00000000004004f5 <+8>:     movl   $0x3,-0x4(%rbp)
   0x00000000004004fc <+15>:    mov    $0x2,%esi
   0x0000000000400501 <+20>:    mov    $0x1,%edi
   0x0000000000400506 <+25>:    callq  0x40050d <func1>
   0x000000000040050b <+30>:    leaveq
   0x000000000040050c <+31>:    retq
End of assembler dump.

(gdb) x/20gx $rbp-0x30
0x7fffffffe340: 0x00007ffff7ffe1c8      0x0000000000000000
0x7fffffffe350: 0x0000000000000001      0x0000000100000002
0x7fffffffe360: 0x00007fffffffe390      0x0000000000000000
0x7fffffffe370: 0x00007fffffffe390      0x000000000040050b
0x7fffffffe380: 0x00007fffffffe470      0x0000000300000000
0x7fffffffe390: 0x0000000000000000      0x00007ffff7a32f45
0x7fffffffe3a0: 0x0000000000000000      0x00007fffffffe478
0x7fffffffe3b0: 0x0000000100000000      0x00000000004004ed
0x7fffffffe3c0: 0x0000000000000000      0x8aa9ec9f71b411ab
0x7fffffffe3d0: 0x0000000000400400      0x00007fffffffe470
```

3. GNU make:-

Compilation of multiple source files is possible in 2 ways.
1. Manual Method.
2. Automation Method: uses GNU Make tool.

What is Makefile?

- Makefile help to compile multiple source files.
- Makefile is Dependency tracking utility.
- Make file look in to the current directory for a file by the name Makefile (recommended) or makefile.

Makefile Syntax:
Target: Dependencies
<tab> Commands

```
# simple make file

all: main

main: main.o add.o sub.o
    gcc -o main main.o sub.o add.o

main.o: main.c
    gcc -c main.c

add.o: add.c
    gcc -c add.c

sub.o: sub.c
    gcc -c sub.c

clean:
    rm –rf *.o
```

Make Advantages:

- Make is a utility for automatically building executable programs from source code.
- Makefile compiles only modified files based on compilation time (i.e., time stamp) because of this compilation time is reduced.

Makefile Examples:
source files: main.c,add.c,sub.c
header files: myinclude.h
Example 1: This is Basic example of Makefile.
$ make (default target is all)
Example 2: This example shows how to use, clean and install targets, in Makefile.
$ make clean (This command cleans object files, binaries and configuration files)
$ make install (This command copy the binaries in /usr/bin directory)
Example 3: This Example shows how to use environment variables in Makefile
Example 4: This Example shows how to create multiple Makefiles in each directory.
Example 5: This Example shows how to give user defined name to Makefile

Static Linker vs Dynamic Linker:
Executale and Linkable Format (ELF) is a standard binary file format.
$ readelf -a first | more (readelf is a details of ELF file)

creation of exec file using Dynamic linker
$ gcc first.c -o first (by default)
creation of exec file using Static linker
$ gcc -static first.c -o first

Static Library vs Dynamic Library:

- Size is Different.

- Static Linkers use static libraries which are appended the executable image at build time.
- Dynamic Linkers use dynamic libraries which carrys symbolic reference in exectable and physically loaded at runtime.

Procedure for Creation of static libraries:

Step1: Implementaion of Source code.
one.c
two.c

Step2: Compile source code up to object file.
$ gcc -c one.c two.c

Step3: Use UNIX archive tools create library image.
$ ar -rcs libourown.a one.o two.o

Procedure for Creation of Dynamic libraries (Shared Libraries):

Step1: Implementaion of Source code.
one.c
two.c

Step2: Compile source to create position independent relocatale.
$ gcc -c -fpic one.c
$ gcc -c -fpic two.c

Step3:
$ gcc -shared -o libourown.so one.o two.o

Telling GCC where to find the User defiend header file:
-I <USER DEFINED HEADER PATH>

Telling GCC where to find the shared library:

Why -l<library name>, -L and LD_LIBRARY_PATH?

-l<library name> means compiler search the library
-L <path> means gcc compiler checks libraries at compilation time in path location.

Making the library available at runtime:
LD_LIBRARY_PATH=<path> means binary execution time checks the library PATH.

$ file libourown.a
libourown.a: current ar archive
$ file libourown.so
libourown.so: ELF 32-bit LSB shared object, Intel 80386, version 1 (SYSV), dynamically linked, not stripped

Examples:
libraries/addsub_lib: how to create static and dynamic libraries.

4. GNU BinUtils:

ar: Creates static libraries, and inserts, deletes, lists, and extracts members.
strings: Lists all of the printable strings contained in an object file.
strip: Deletes symbol table information from an object file.
nm: Lists the symbols defined in the symbol table of an object file.
size: Lists the names and sizes of the sections in an object file.
readelf: Displays the complete structure of an object file, including all of the information encoded in the ELF header; subsumes the functionality of size and nm.
objdump: The mother of all binary tools. Can display all of the information in an object file. Its most useful function is disassembling the binary instructions in the .text section.
ldd: Lists the shared libraries that an executable needs at run time

5. Mini Project :

1. Assume that you are working on a calculator project contained in the directory calculator. under this directory there are several subdirectories with names "lib","doc","include“,"src“ and “bin”.
2. The objective is to create a top level Makefile in the calculator directory and each of the source code directories: lib and src.
3. The include directory contains the header files for the project and does not need a Makefile as these don't have to be compiled separately.
4. When make is invoked in the calculator directory it should automatically invoke the Makefile in each of the src directories automatically and build a library in the "lib" directory and two executables in each of "src" directory.
5. Create any source files with all their header files in the include directory.
6. **GNU Build System supports the following configurations:**
    - DEBUG_BUILD; STATIC_BUILD; ADD ; SUB; MUL ; DIV;